6ELEN018W - Tutorial 4 Exercises

Exercise 1

Consider the gridworld problem shown in Figure 1. An agent needs to move to the target goal position **G** starting from any cell (arbitrary position) by following the optimum policy which gathers the largest reward.

The immediate rewards $r(s, a)$ for the transition from one state to another for this problem are also shown in Figure 1. For example, the reward received by moving from state s_1 to s_2 by taking the *right* action is 0. The reward for moving from state s_5 to state G (i.e. taking the *north* action) is 100.

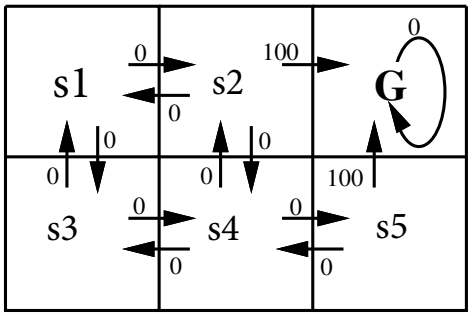


Figure 1: $r(s, a)$ (immediate reward) values.

Write a Python program to estimate the V* values based on a modified version of formula (1) of the lecture which includes a discount factor $\gamma = 0.9$:

$$V(S_t) \leftarrow V(S_t) + \alpha[\gamma V(S_{t+1}) - V(S_t)] \tag{1}$$

where $V(S_{t+1})$ is the next state after taking a greedy action from state $V(S_t)$

Hint: You should follow the pseudocode in Slide 8. Start with initial values of 0, except that state s3 (the Goal) which has a value of 100.

Display all the estimated V* values and compare them with the values shown in Figure 2 (Lecture Slide 6).

Do not look at the solution provided until you spend sufficient time working out your own solution.

Exercise 2

Extend your code so that the robot can choose the optimum actions based on the estimated V values. Write a Python function which accepts a single state as an argument and returns the full path to the goal position G, by taking the optimum actions at each time step, based on your estimated V values.

Sample Solution

```
import random

# delta (see lecture slides formulas) is a dictionary containing
# the states of the dynamic system and mapping them to a dictionary
# {a: s'} where s' is the new state that we arrive when we apply
# action a when in state s

# We assume that states, s_1, s_2 to s_6 are assigned by row traversing:
#-----------
# |s1 |s2 |s3|
# |s4 |s5 |s6|
#-----------
delta = {}

# 'N' is go North action
# 'E' is go East action
# 'S' is go South action
# 'W' is go West action
delta['s1'] = {'N':'s1', 'E':'s2', 'S':'s4', 'W':'s1'}
delta['s2'] = {'N':'s2', 'E':'s3', 'S':'s5', 'W':'s1'}
delta['s3'] = {'N':'s3', 'E':'s3', 'S':'s6', 'W':'s2'}
delta['s4'] = {'N':'s1', 'E':'s5', 'S':'s4', 'W':'s4' }
delta['s5'] = {'N':'s2', 'E':'s6', 'S':'s5', 'W':'s4'}
delta['s6'] = {'N':'s3', 'E':'s6', 'S':'s6', 'W':'s5'}
# all rewards are zero, except those leading to state s3

V = {}
V['s1'] = 0
V['s2'] = 0
V['s3'] = 100
V['s4'] = 0
V['s5'] = 0
V['s6'] = 0


# action 1 is North
# action 2 is East
# action 3 is South
# action 4 is West
def num2Action(action):
    if action == 1:
        a = 'N'
    elif action == 2:
        a = 'E'
    elif action == 3:
        a = 'S'
    else:
        a = 'W'
```

```
    return a


# accepts a single state and returns the greedy action based on V values
def find_greedy_action(s):
    max_so_far = 0
    # if all actions are equally good (or bad) choose a random one
    if V[delta[s]['N']] == V[delta[s]['W']] and
       V[delta[s]['W']] == V[delta[s]['E']] and
       V[delta[s]['E']] == V[delta[s]['S']]:
           max_action = random.randint(1, 4)
           max_action = num2Action(max_action)
    else:
        max_so_far = V[delta[s]['N']]
        max_action = 'N'

    if V[delta[s]['E']] > max_so_far:
        max_so_far = V[delta[s]['E']]
        max_action = 'E'

    if V[delta[s]['S']] > max_so_far:
        max_so_far = V[delta[s]['S']]
        max_action = 'S'

    if V[delta[s]['W']] > max_so_far:
        max_so_far = V[delta[s]['W']]
        max_action = 'W'

    return max_action


# display Q values
def displayV():
    print('s1:-> ', V['s1'])
    print('s2:-> ', V['s2'])
    print('s3:-> ', V['s3'])
    print('s4:-> ', V['s4'])
    print('s5:-> ', V['s5'])
    print('s6:-> ', V['s6'])


 # apply V-learning
def V_learning():
    alpha = 0.5
    e = 0.1
    gamma = 0.9

    # Run many episodes
    for episode in range(100):
```

```
        # Run 1 episode of the problem (i.e. until state 'G' is reached)
        # Start with from random state s
        s = random.randint(1, 6)
        s = 's' + str(s) # current state

        while s != 's3':
            # 1. Select an action and execute it based on e-greedy approach (e=0.1)
            # generate a random number [0,1]
            r = random.random()

            if r < (1-e):  # greedy
                greedy = True
                a = find_greedy_action(s)
            else:
                greedy = False
                a = random.randint(1, 4)
                a = num2Action(a)

            # 3. Observe the new state s'
            s_new = delta[s][a]
            #if __debug__:
            #    print(s, '(', a, ')', '->', s_new)

            # 3b. Receive immediate reward r
            # if next state is the goal state G then the reward is 100,
            # otherwise it is 0 - this is not used in this algorithm
            # as V(G) is fixed to 100
            if s_new == 's3':
                r = 100
            else:
                r = 0

            # 4. Update the table entry if a greedy action was taken based on equation (1):
            #
            if s_new != s and greedy:
                a_next = find_greedy_action(s_new)
                V[s] = V[s] + alpha*(gamma*V[s_new] - V[s])

            # 5. s <- s'
            # The new state becomes the next current state
            s = s_new

    displayV()


# run the code
V_learning()
```

The output of the program is:

```
s1:->  80.99999999727814
s2:->  90.0
s3:->  100
s4:->  72.89912693387618
s5:->  80.99958192684397
s6:->  89.97802734375
```